Deterministic QMC Methodology in **mvtnorm**

The **mvtnorm** authors

May 3, 2026

1 The computational problem

The functions `pmvnorm` and `pmvt` evaluate probabilities over rectangular regions for multivariate normal and Student t distributions. After standardisation, the central numerical problem is the evaluation of an integral over a hyperrectangle with dependence induced by a correlation matrix. For dimensions beyond the very small cases, direct tensor-product quadrature is not competitive because its cost grows exponentially with dimension.

The default **mvtnorm** algorithm is the Genz-Bretz randomized quasi-Monte Carlo (QMC) method (Genz, 1992; Genz and Bretz, 1999, 2002, 2009). This method transforms the original probability into an integral over the unit cube and estimates that integral by using low-discrepancy points with randomization. The randomization is useful: it gives an empirical Monte Carlo error assessment and, in practice, high speed for many medium- and high-dimensional problems.

The new `DeterministicQMC` option keeps the same general transformed integration viewpoint, but removes dependence on R's random-number generator. The plain C backends implement normal probabilities by combining a Cholesky factorization and the sequential conditional transformation used in Genz type algorithms. The `"c"` backend uses a deterministic Halton rule; the `"sobol"` backend uses a deterministic Sobol digital net; the `"hybrid"` backend tests a Sobol rule with a deterministic tail-spaced change of variables; the `"genzsobol"` backend adds marginal-width variable reordering and embedded stability checks. The original FORTRAN code is retained as a reference and as a fallback, including for Student t probabilities in the current implementation. The default deterministic choice, `"auto"`, now selects a cross-checked Genz-Sobol policy for supported normal problems and keeps the deterministic FORTRAN path for higher-dimensional normal problems or Student t probabilities.

2 Algorithms available in this comparison

Algorithm	Main advantages	Main limitations
`GenzBretz` default	Fast randomized QMC; supports normal and t probabilities; has a randomized error estimate; is the mature default for general use.	Repeated calls can differ unless the seed is controlled; stochastic noise can be inconvenient inside deterministic optimization or regression tests.
`Miwa`	Deterministic; often highly accurate for low-dimensional normal probabilities; does not depend on random seeds.	Limited to normal probabilities and dimensions up to 20; cost grows quickly with dimension and the number of grid steps.
Original fixed-budget QMC	Uses the established Genz-Bretz randomized QMC implementation with early stopping disabled, making the computational budget easier to compare.	Still randomized; reported error is tied to randomized replication; speed is budget dependent.
`DeterministicQMC`	Deterministic output without managing R's RNG state; plain C Halton, Sobol, hybrid, and Genz-Sobol normal backends; FORTRAN reference/fallback remains available; scales by a point budget rather than by a grid in each dimension.	The C backends currently handle normal probabilities only; the deterministic error values are convergence diagnostics, not randomized confidence statements; the implementation is intentionally conservative and does not yet include all adaptations from the mature FORTRAN path.

3 What is new

The new backend is not a replacement for the original FORTRAN routines. It is a second deterministic route through the same **mvtnorm** interface:

- It introduces a plain C implementation for multivariate normal prob-

abilities, callable through `DeterministicQMC(backend = "c")` for a Halton rule and `DeterministicQMC(backend = "sobol")` for Sobol direction numbers with Gray-code point generation.

- It includes an experimental `DeterministicQMC(backend = "hybrid")` path that uses the Sobol generator with a deterministic tail-spaced change of variables and Jacobian correction. This is intended to test whether sparse tail spacing improves fixed-budget accuracy.
- It includes `DeterministicQMC(backend = "genzsobol")`, which adds two algorithmic pieces from the proposed fuller backend: variables are reordered by marginal probability width before the Cholesky factorization, and estimates are checked at embedded powers of two against `abseps` and `releps`.
- It leaves the FORTRAN implementation in place as a reference and fallback. This is useful both scientifically and practically: numerical changes can be checked against the historical implementation.
- It adds `DeterministicQMC(backend = "auto")` as a best-practice deterministic selector that uses cross-checked Genz-Sobol when supported and deterministic FORTRAN when the C Sobol path is not available.
- It separates determinism from the Miwa algorithm. Miwa's method is deterministic because it is a low-dimensional grid algorithm; the new method is deterministic because the integration points are fixed. Thus the new method can be used with a budget such as `maxpts = 25000`, just as randomized QMC is used.
- It gives deterministic, seed-independent results, which is helpful for test suites, simulation studies that need exact reproducibility, and objective functions used by optimizers.

The main advantage over the previous deterministic option, `Miwa`, is therefore not that it is universally more accurate. Rather, the new method is designed to give deterministic QMC behavior under the same kind of point-budget control as `GenzBretz`. This makes it attractive when reproducibility is more important than randomized error assessment, especially as dimension grows beyond the range where a grid method is comfortable.

4 Backend choices

The constructor exposes backend choices explicitly:

```
> GenzBretz()

$maxpts
[1] 25000

$abseps
[1] 0.001

$releps
[1] 0

attr(,"class")
[1] "GenzBretz"

> DeterministicQMC(backend = "auto")

$maxpts
[1] 25000

$abseps
[1] 0.001

$releps
[1] 0

$backend
[1] "auto"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> DeterministicQMC(backend = "c")

$maxpts
[1] 25000

$abseps
[1] 0.001
```

```
$releps
[1] 0

$backend
[1] "c"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> DeterministicQMC(backend = "sobol")

$maxpts
[1] 25000

$abseps
[1] 0.001

$releps
[1] 0

$backend
[1] "sobol"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> DeterministicQMC(backend = "hybrid")

$maxpts
[1] 25000

$abseps
[1] 0.001
```

```
$releps
[1] 0

$backend
[1] "hybrid"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> DeterministicQMC(backend = "genzsobol")

$maxpts
[1] 25000

$abseps
[1] 0.001

$releps
[1] 0

$backend
[1] "genzsobol"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> DeterministicQMC(backend = "fortran")

$maxpts
[1] 25000

$abseps
[1] 0.001

$releps
```

```
[1] 0

$backend
[1] "fortran"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"

> Miwa()

$steps
[1] 128

$checkCorr
[1] TRUE

$maxval
[1] 1000

attr(,"class")
[1] "Miwa"
```

With `backend = "auto"`, supported normal probabilities use the C Genz-Sobol backend and Student t probabilities use the deterministic FORTRAN fallback. More specifically, the C path uses several fixed deterministic sequence offsets; the reported error is the larger of the embedded convergence errors and the between-check spread. The `"cpu"` alias maps to `"c"`. The `"metal"` option currently probes for a macOS Metal device but falls back to C/FORTRAN because no Metal evaluator is implemented yet.

5 The Genz-Sobol backends and the faster target

The first C backend deliberately used a simple deterministic Halton rule. The new `backend = "sobol"` implements the next point-generator step: it keeps the sequential Genz transformation and replaces Halton points with an embedded Sobol digital net generated from direction numbers in plain C. The experimental `backend = "hybrid"` keeps that Sobol backbone and applies a

gentle deterministic tail-spaced warp with a Jacobian adjustment. This lets the benchmark test the nonuniform-grid idea without replacing QMC by a tensor grid. The `backend = "genzsobol"` option keeps the Sobol backbone, reorders variables before factorization, and can stop early when embedded powers-of-two estimates stabilize.

The backend is still intentionally conservative. It does not yet include all of the refinements that make the mature randomized Genz-Bretz path so effective, but `backend = "auto"` now implements the first automatic policy: run Genz-Sobol at several fixed deterministic sequence offsets, combine the estimates, and use the larger of embedded convergence error and between-check spread as the accuracy diagnostic. When a tolerance is requested, the repeated checks are run against a tighter internal target before the combined diagnostic is compared with the user-requested tolerance. The remaining speed-and-accuracy target is to add:

- reorder variables before integration so that the transformed integrand has lower effective dimension;
- compare the Sobol backend with the experimental tail-spaced hybrid;
- replace the present fixed sequence offsets by stronger deterministic digital shifts or rank-1 lattice shifts while preserving exact reproducibility;
- adaptively double the point budget until embedded estimates stabilize;
- fall back to the deterministic FORTRAN reference path, or to a larger budget, when the deterministic stability diagnostic is poor.

The Sobol backend does not have the same randomized error interpretation as `GenzBretz`, because that interpretation comes from randomization. Its goal is instead comparable numerical accuracy with deterministic output. This is plausible because it uses the same Genz-style conditioning strategy and a stronger low-discrepancy design than plain Halton; further effective-dimension reduction remains a natural enhancement. The hybrid is useful to test because it preserves the Sobol backbone while borrowing the best part of nonuniform grids: not all tail regions deserve the same local resolution. The `genzsobol` backend is the first implemented version that uses reordering and embedded deterministic stopping together, while `backend = "auto"` adds the outer deterministic cross-check policy that makes the error attribute more useful in unattended workflows.

6 Accuracy requests and warning

Users can request diagnostic error control with the familiar `abseps` and `releps` arguments:

```
> DeterministicQMC(maxpts = 100000, abseps = 1e-6, releps = 0,
+                  backend = "auto")

$maxpts
[1] 1e+05

$abseps
[1] 1e-06

$releps
[1] 0

$backend
[1] "auto"

$start
[1] 0

attr(,"class")
[1] "DeterministicQMC" "GenzBretz"
```

For `DeterministicQMC`, this is a deterministic stability target. If the reported diagnostic is larger than the requested absolute or relative tolerance after the available `maxpts` budget is spent, the result is returned with `inform = 1` and the message `"Completion with error > abseps"`. This is useful for unattended deterministic workflows because the caller can detect that the reproducible diagnostic did not certify the requested target.

The warning is that this diagnostic is not the same object as the randomized Monte Carlo error reported by `GenzBretz`. It is not a confidence interval, and it is not a rigorous deterministic error bound. It should be read as a reproducible convergence and stability check.

7 Practical interpretation

For exploratory probability calculations, `GenzBretz` remains the best default because it is mature, fast, and supplies an error estimate based on random-

ization. For low-dimensional normal probabilities, `Miwa` remains valuable as a deterministic check. The new `DeterministicQMC` backend is most useful when exact reproducibility of the numerical path is required and the user wants a QMC-like budget rather than a low-dimensional grid.

The deterministic error reported by `DeterministicQMC` should be read as a stability diagnostic across deterministic blocks. It is not the same object as the randomized Monte Carlo error reported by `GenzBretz`.

References

Genz A (1992). "Numerical Computation of Multivariate Normal Probabilities." *Journal of Computational and Graphical Statistics*, **1**(2), 141–149. doi:10.1080/10618600.1992.10477010.

Genz A, Bretz F (1999). "Numerical Computation of Multivariate t-probabilities with Application to Power Calculation of Multiple Contrasts." *Journal of Statistical Computation and Simulation*, **63**(4), 103–117. doi:10.1080/00949659908811962.

Genz A, Bretz F (2002). "Methods for the Computation of Multivariate t Probabilities." *Journal of Computational and Graphical Statistics*, **11**(4), 950–971. doi:10.1198/106186002394.

Genz A, Bretz F (2009). *Computation of Multivariate Normal and t Probabilities*. Lecture Notes in Statistics. Springer-Verlag, Heidelberg, Germany. ISBN 978-3-642-01688-2.